MODUL 01 · BAB I · DASAR ALGORITMA

Konsep Dasar Algoritma: Definisi, Karakteristik, Representasi, dan Struktur

Fondasi untuk seluruh modul: apa itu algoritma, syarat algoritma yang baik, cara menuliskannya, dan tiga struktur pembangunnya.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan pengertian algoritma, program, dan pemrograman serta hubungan ketiganya.
- Mengidentifikasi enam karakteristik algoritma dan mengenali algoritma yang melanggarnya.
- Menuliskan algoritma dalam bentuk naratif, flowchart, dan pseudocode.
- Membedakan struktur runtunan, percabangan, dan perulangan, lalu menelusuri (*trace*) algoritma sederhana langkah demi langkah.

PRASYARAT

- Tidak ada prasyarat khusus. Modul ini adalah titik awal.

BAB / MATERI ASAL

PENGERTIAN ALGORITMA
PEMROGRAMAN (bagian A–C)

POSISI DALAM URUTAN BELAJAR

Modul pertama. Menjadi dasar Modul 02 (Iteratif & Rekursif) dan Modul 03 (Running Time).

DAFTAR ISI

1. Mengapa Kita Perlu Memahami Algoritma? 1
2. Pengertian Algoritma, Program, dan Pemrograman 2
3. Karakteristik Algoritma 3
4. Tiga Cara Menuliskan Algoritma 5
5. Struktur Dasar Algoritma 7
6. Menelusuri Algoritma Langkah demi Langkah 8
7. Bagaimana Konsep Ini Terhubung dengan Modul Lain 10
8. Kesimpulan 10
- Kuis Pemahaman 12
- Kunci Jawaban dan Pembahasan 13
- Referensi 14

PENDAHULUAN

1. Mengapa Kita Perlu Memahami Algoritma?

Setiap kali kamu mencari kontak di ponsel, memakai aplikasi peta, atau mengurutkan berkas berdasarkan tanggal, ada serangkaian langkah yang dijalankan komputer secara tepat dan berurutan. Serangkaian langkah itulah yang disebut **algoritma**. Komputer sangat cepat, tetapi ia tidak bisa menebak maksud kita. Ia hanya mengerjakan perintah yang diberikan, tepat seperti tertulis.

Karena itu, sebelum menulis program, kita perlu mampu merancang langkah penyelesaian masalah yang benar, jelas, dan efisien. Mata kuliah *Perancangan dan Analisis Algoritma* membahas dua pertanyaan besar:

bagaimana merancang algoritma untuk suatu masalah, dan **bagaimana menganalisis** apakah algoritma tersebut benar dan seberapa cepat ia berjalan.

Modul ini meletakkan fondasinya. Kita mulai dari pengertian algoritma dan kaitannya dengan program, lalu membahas karakteristik yang wajib dimiliki sebuah algoritma, tiga cara menuliskannya (naratif, flowchart, pseudocode), dan tiga struktur dasar yang membangun hampir semua algoritma. Di akhir modul, kamu akan menelusuri dua algoritma sederhana langkah demi langkah, sebuah keterampilan yang terus dipakai di semua modul berikutnya.

PEMBAHASAN INTI

2. Pengertian Algoritma, Program, dan Pemrograman

2.1 Asal-usul istilah

Kata *algoritma* berasal dari nama ilmuwan Persia abad ke-9, **Muhammad ibn Musa al-Khwarizmi**, yang menulis karya tentang prosedur sistematis untuk menyelesaikan perhitungan dan persamaan. Namanya, setelah dilatinkan menjadi *Algoritmi*, kemudian dipakai untuk menyebut prosedur perhitungan yang langkah-langkahnya tertentu. Pada tahun 1936, **Alan Turing** merumuskan model mesin abstrak, yang kini dikenal sebagai *mesin Turing*, untuk menjelaskan secara matematis apa artinya sebuah prosedur dapat dijalankan secara mekanis. Gagasan ini, yang terbit sebagai makalah Turing (1937), menjadi landasan teoretis komputer modern: apa pun yang dapat diselesaikan dengan algoritma, pada prinsipnya dapat dikerjakan oleh mesin.

2.2 Definisi algoritma

DEFINISI

Algoritma adalah urutan langkah logis dan terbatas untuk menyelesaikan suatu masalah. Dalam konteks komputasi, algoritma adalah urutan langkah komputasi yang mengubah **masukan (input)** menjadi **keluaran (output)** yang diinginkan.

Definisi ini menekankan tiga hal. Pertama, ada **masalah** yang ingin diselesaikan, misalnya "urutkan daftar nilai mahasiswa". Kedua, ada **langkah-langkah** yang dikerjakan secara berurutan. Ketiga, langkah-langkah itu menghasilkan **keluaran** yang memiliki hubungan jelas dengan masukannya. Cormen dkk. (2009) menyebut sebuah algoritma *benar* (*correct*) apabila untuk **setiap** masukan yang sah, algoritma tersebut berhenti dan memberikan keluaran yang benar.

Perhatikan kata "setiap". Sebuah algoritma pengurutan yang hanya benar untuk data yang sudah hampir urut bukanlah algoritma pengurutan yang benar. Cukup satu masukan yang membuatnya salah, algoritma itu dianggap tidak benar.



Gambar 1. Algoritma sebagai jembatan dari masukan ke keluaran. Contoh: masalah pengurutan.

2.3 Program dan pemrograman

Program adalah kumpulan instruksi, ditulis dalam bahasa pemrograman tertentu, yang mengikuti suatu algoritma agar komputer dapat mengerjakan tugas tertentu. **Pemrograman** adalah proses membuat program tersebut. Prosesnya mencakup menulis kode, menguji program, melakukan *debugging* (mengidentifikasi dan menghilangkan *bug* atau kesalahan di dalam kode), hingga menghasilkan solusi yang dapat dijalankan komputer.

Hubungan keduanya dapat diringkas begini: **algoritma menjawab "apa yang harus dilakukan"**, sedangkan **pemrograman menerjemahkan jawaban itu ke dalam bahasa yang dimengerti komputer**. Satu algoritma yang sama dapat diterjemahkan ke banyak bahasa (Python, C, Java) tanpa mengubah logikanya.

Aspek	Algoritma	Program	Pemrograman
Hakikat	Rancangan langkah logis	Wujud konkret algoritma dalam bahasa pemrograman	Aktivitas membuat program
Bergantung bahasa?	Tidak	Ya (Python, C, Java, ...)	Ya
Bisa dijalankan komputer?	Tidak secara langsung	Ya	-
Contoh	"Bandingkan setiap bilangan dengan nilai maksimum sementara"	`if x > maks: maks = x`	Menulis, menguji, dan memperbaiki kode di atas

Tabel 1. Perbedaan algoritma, program, dan pemrograman.

PEMBAHASAN INTI

3. Karakteristik Algoritma

Tidak semua daftar langkah layak disebut algoritma. Bahan kuliah menyebutkan enam karakteristik penting. Lima di antaranya identik dengan lima sifat algoritma yang dirumuskan Knuth (1997): *finiteness*, *definiteness*, *input*, *output*, dan *effectiveness*. Karakteristik keenam, *structured*, adalah tambahan yang menekankan kerapian penyusunan langkah.

3.1 Finiteness (keterbatasan langkah)

Algoritma harus **berhenti** setelah sejumlah langkah yang terbatas. Program yang tidak pernah berhenti menandakan algoritmanya salah. Contoh pelanggaran yang sering terjadi: perulangan yang kondisinya tidak pernah menjadi salah.

Contoh algoritma yang TIDAK memenuhi finiteness

```
i = 1
while i > 0
    i = i + 1      // i selalu positif, perulangan tidak pernah berhenti
```

3.2 Input (masukan)

Algoritma memiliki **nol atau lebih** masukan, yaitu nilai yang diberikan sebelum algoritma dijalankan. "Nol" berarti ada algoritma yang tidak memerlukan masukan sama sekali, misalnya algoritma yang mencetak sepuluh bilangan prima pertama.

3.3 Output (keluaran)

Algoritma memiliki **satu atau lebih** keluaran yang berhubungan dengan masukannya, yaitu solusi dari masalah. Contoh: algoritma menghitung luas segitiga dengan masukan alas 8 dan tinggi 6 harus menghasilkan keluaran $\frac{1}{2} \times 8 \times 6 = 24$. Algoritma tanpa keluaran tidak berguna, karena tidak ada hasil yang bisa diamati.

3.4 Definiteness (kepastian)

Setiap langkah harus didefinisikan secara **tepat dan tidak ambigu**. Perintah "tambahkan sedikit garam" bukan langkah algoritma yang pasti, sedangkan "tambahkan 5 gram garam" sudah pasti. Dalam pemrograman, kepastian juga berarti kita tahu jenis data yang diolah. Misalnya, masukan untuk menghitung luas segitiga adalah bilangan, bukan teks.

> **CONTOH DARI BAHAN KULIAH: MENGHITUNG BANYAKNYA BILANGAN**
>
> Untuk menghitung banyaknya bilangan dari n bilangan yang dimasukkan, langkahnya harus dinyatakan secara pasti:
>
> 1. Buat variabel `jumlahbil` dengan nilai awal nol (`jumlahbil = 0`).
> 2. Setiap kali satu bilangan dibaca, lakukan `jumlahbil = jumlahbil + 1`.
>
> Kalimat "hitung saja bilangannya" tidak memenuhi *definiteness* karena tidak menjelaskan nilai awal maupun cara menambahkannya.

3.5 Effectiveness (keefektifan)

Setiap langkah harus cukup sederhana sehingga dapat dikerjakan dengan tepat dalam waktu yang wajar. Algoritma juga harus sesuai dengan masalahnya dan benar-benar menghasilkan solusi. Bahan kuliah menambahkan bahwa algoritma yang baik juga **efisien**: waktu prosesnya relatif singkat dan penggunaan memorinya sedikit.

KLARIFIKASI ISTILAH: EFEKTIF VS EFISIEN

Dalam rumusan Knuth (1997), *effectiveness* berarti setiap operasi cukup dasar sehingga, pada prinsipnya, dapat dikerjakan secara tepat dalam waktu terbatas, bahkan oleh manusia dengan kertas dan pensil. **Efisiensi** adalah konsep yang berbeda: seberapa hemat algoritma memakai waktu dan memori. Sebuah algoritma bisa efektif tetapi tidak efisien, misalnya mencoba semua kemungkinan urutan untuk mengurutkan data. Bahan kuliah menggabungkan keduanya dalam satu poin. Di modul-modul berikutnya, efisiensi akan dibahas tersendiri melalui analisis *running time* (Modul 03–07).

3.6 Structured (terstruktur)

Langkah-langkah harus disusun secara sistematis sehingga tidak berbelit-belit. Bagian masukan, proses, dan keluaran dapat dibedakan dengan jelas. Algoritma yang terstruktur mudah diperiksa ulang, mudah diubah, dan mudah diterjemahkan menjadi program.

Karakteristik	Syarat	Contoh pelanggaran
Finiteness	Berhenti setelah langkah terbatas	Perulangan `while i > 0` dengan `i` yang terus bertambah
Input	Nol atau lebih masukan yang jelas	Memakai variabel yang tidak pernah diberi nilai
Output	Satu atau lebih keluaran	Menghitung hasil tetapi tidak pernah mengembalikannya
Definiteness	Setiap langkah pasti, tidak ambigu	"Ulangi sampai cukup"
Effectiveness	Langkah dapat dikerjakan dan menghasilkan solusi	"Tuliskan semua digit bilangan π"
Structured	Tersusun sistematis	Langkah melompat-lompat tanpa urutan yang jelas

Tabel 2. Ringkasan enam karakteristik algoritma dan contoh pelanggarannya.

PEMBAHASAN INTI

4. Tiga Cara Menuliskan Algoritma

Algoritma yang sama dapat disajikan dalam beberapa bentuk. Bahan kuliah membahas tiga bentuk. Untuk membandingkannya, kita pakai satu masalah yang sama: **menentukan bilangan terbesar dari tiga bilangan sembarang**.

4.1 Naratif (bahasa alami)

Algoritma ditulis dengan bahasa manusia sehari-hari. Bentuk ini mudah dibaca siapa saja, tetapi rawan ambigu karena bahasa alami sering bermakna ganda.

1. Ambil bilangan pertama dan tetapkan `maks` sama dengan bilangan pertama.
2. Ambil bilangan kedua dan bandingkan dengan `maks`. Apabila bilangan kedua lebih besar dari `maks`, tetapkan `maks` sama dengan bilangan kedua.
3. Ambil bilangan ketiga dan bandingkan dengan `maks`. Apabila bilangan ketiga lebih besar dari `maks`, tetapkan `maks` sama dengan bilangan ketiga.

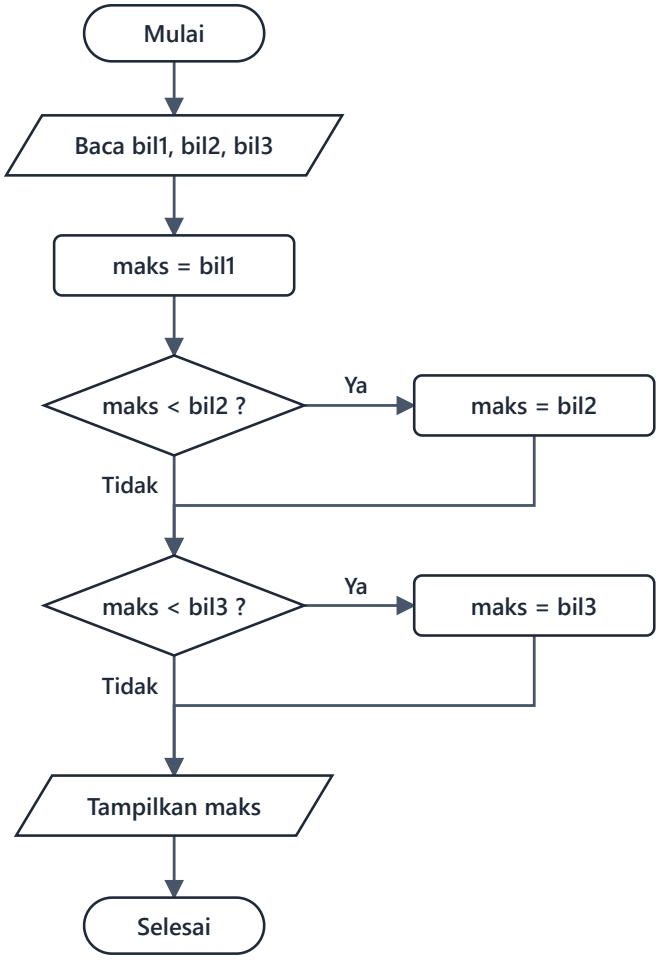
4 Variabel `maks` kini berisi bilangan terbesar. Tampilkan hasilnya.

4.2 Flowchart (diagram alir)

Flowchart menyajikan algoritma sebagai simbol-simbol grafis yang dihubungkan anak panah sesuai arah aliran proses. Bentuk ini mudah dipahami secara visual, tetapi merepotkan bila algoritmanya panjang.

Simbol	Nama	Fungsi
Oval / kapsul	Terminal	Menandai awal (Mulai) dan akhir (Selesai)
Jajaran genjang	Input/Output	Membaca masukan atau menampilkan keluaran
Persegi panjang	Proses	Perhitungan atau pemberian nilai, misalnya `maks = bil1`
Belah ketupat	Keputusan	Pemeriksaan kondisi dengan cabang Ya/Tidak
Anak panah	Garis alir	Menunjukkan urutan eksekusi

Tabel 3. Simbol flowchart yang paling sering dipakai.



Gambar 2. Flowchart untuk menentukan bilangan terbesar dari tiga bilangan (digambar ulang dari bahan kuliah).

Perhatikan bahwa pada flowchart di atas, kedua cabang "Tidak" langsung turun ke pemeriksaan berikutnya, sedangkan cabang "Ya" mengubah nilai `maks` terlebih dahulu sebelum bergabung kembali. Inilah wujud visual dari struktur percabangan.

4.3 Pseudocode

Pseudocode menuliskan langkah-langkah dengan gaya yang mirip bahasa pemrograman, tetapi tidak terikat pada satu bahasa tertentu. Karena itu pseudocode mudah diterjemahkan ke bahasa apa pun. Kelemahannya, pseudocode lebih sulit dipahami oleh orang yang belum pernah belajar pemrograman. Pseudocode adalah bentuk yang paling banyak dipakai dalam modul-modul ini.

Konvensi pseudocode yang dipakai dalam bahan kuliah (dan buku Cormen dkk.) adalah sebagai berikut:

- Tanda `=` berarti **pemberian nilai** (*assignment*). Contoh: `max = bil2` artinya "isi `max` dengan nilai `bil2`".
- Tanda `==` berarti **pemeriksaan kesamaan** di dalam kondisi `if` atau `while`. Contoh: "jika n modulo 2 sama dengan 0" ditulis `if (n mod 2 == 0)`.
- Pernyataan yang menjadi bagian dari perulangan atau dari cabang kondisi ditulis **menjorok ke dalam** (indentasi). Indentasi menggantikan tanda kurung kurawal.
- Teks setelah `//` adalah komentar, bukan perintah.

BILANGAN_MAX(bil1, bil2, bil3)

```
1  max = bil1
2  if (max < bil2)
3      then max = bil2
4  if (max < bil3)
5      then max = bil3
6  return max
```

`BILANGAN_MAX` adalah nama fungsi (atau *method*), sedangkan `bil1`, `bil2`, `bil3` di dalam tanda kurung adalah **parameter masukan**. Perintah `return` mengembalikan keluaran kepada pemanggil fungsi.

Bentuk	Kelebihan	Kekurangan	Cocok untuk
Naratif	Mudah dipahami siapa saja	Sering ambigu, panjang	Menjelaskan ide awal
Flowchart	Visual, alur percabangan terlihat jelas	Repot dibuat untuk algoritma panjang	Algoritma pendek, presentasi
Pseudocode	Ringkas, presisi tinggi, mudah diterjemahkan ke kode	Perlu latar belakang pemrograman	Analisis dan implementasi

Tabel 4. Perbandingan tiga cara menuliskan algoritma.

PEMBAHASAN INTI

5. Struktur Dasar Algoritma

Struktur algoritma adalah cara langkah-langkah diorganisasi. Struktur yang baik membuat algoritma efisien, mudah dipahami, dan mudah diubah. Hampir semua algoritma, sekompleks apa pun, dibangun dari tiga struktur berikut.

5.1 Runtunan (sequential)

Langkah dieksekusi satu per satu sesuai urutan penulisan. Ini struktur paling sederhana. Urutan penting: menukar posisi dua langkah dapat mengubah hasil.

Contoh runtunan: menghitung luas segitiga

```
alas = 8
tinggi = 6
luas = alas * tinggi / 2      // luas = 24
return luas
```

5.2 Percabangan (selection)

Algoritma memilih salah satu jalur eksekusi berdasarkan suatu kondisi. Biasanya diwujudkan dengan `if`–`else` atau `switch`. Algoritma `BILANGAN_MAX` adalah contoh struktur percabangan: perintah `max = bil2` hanya dijalankan bila kondisi `max < bil2` bernilai benar.

5.3 Perulangan (iteration / loop)

Sekelompok langkah diulang sampai suatu kondisi terpenuhi. Bentuknya antara lain `for`, `while`, dan `do-while`. Perulangan memungkinkan algoritma yang pendek mengolah data berukuran besar.

JUMLAH_n_BIL(A) // menjumlahkan n bilangan di dalam array A[1..n]

```
jumlah = 0
for i = 1 to n
    jumlah = jumlah + A[i]   // jumlah (kiri) = nilai baru; jumlah (kanan) = nilai lama
return jumlah
```

TENTANG ARRAY (LARIK)

Array adalah kumpulan data sejenis yang disimpan berurutan dan diakses melalui indeks. Pada bahan kuliah ini indeks dimulai dari 1, sehingga array A berukuran $1 \times n$ berisi $A[1], A[2], \ldots, A[n]$. Banyak bahasa pemrograman (termasuk Python) memulai indeks dari 0. Perbedaan ini perlu diingat ketika menerjemahkan pseudocode menjadi program.

CONTOH DAN PENERAPAN

6. Menelusuri Algoritma Langkah demi Langkah

Untuk memastikan sebuah algoritma benar, kita menjalankannya secara manual dengan masukan tertentu lalu mencatat perubahan setiap variabel. Kegiatan ini disebut **penelusuran** (*tracing*). Penelusuran tidak membuktikan kebenaran untuk semua masukan, tetapi sangat efektif untuk menemukan kesalahan dan memahami cara kerja algoritma.

6.1 Penelusuran BILANGAN_MAX

Masukan: `bil1 = 7`, `bil2 = 4`, `bil3 = 10`. Keluaran yang diharapkan adalah 10.

Baris	Perintah	Evaluasi	max
1	`max = bil1`	max diisi 7	7
2	`if (max < bil2)`	7 < 4 bernilai FALSE, baris 3 dilewati	7
4	`if (max < bil3)`	7 < 10 bernilai TRUE, lanjut ke baris 5	7
5	`then max = bil3`	max diisi 10	10
6	`return max`	mengembalikan 10	**10**

Tabel 5. Penelusuran BILANGAN_MAX(7, 4, 10).

Keluaran 10 sesuai harapan. Satu penelusuran belum cukup untuk yakin, jadi sebaiknya kita juga mencoba kasus lain: bilangan terbesar di posisi pertama (misalnya 9, 3, 5), di posisi kedua (2, 8, 1), dan kasus bilangan yang sama (4, 4, 4). Untuk semua kasus tersebut algoritma tetap menghasilkan bilangan terbesar.

6.2 Penelusuran JUMLAH_n_BIL

Masukan: array $A = [20, 35, 15]$ sehingga $n = 3$.

indeks	1	2	3
A	20	35	15

Tabel 6. Isi array A beserta indeksnya.

i	Baris 3: jumlah = jumlah + A[i]	jumlah
-	Baris 1: inisialisasi	0
1	0 + A[1] = 0 + 20	20
2	20 + A[2] = 20 + 35	55
3	55 + A[3] = 55 + 15	70
4	$i = 4 > n$, perulangan berhenti, baris 4 dijalankan	**70**

Tabel 7. Penelusuran JUMLAH_n_BIL untuk A = [20, 35, 15].

Perhatikan bahwa pemeriksaan `for` pada baris 2 dilakukan **empat kali** (untuk $i = 1, 2, 3, 4$), sedangkan isi perulangan (baris 3) hanya dijalankan **tiga kali**. Pemeriksaan keempat adalah saat kondisi perulangan bernilai salah. Detail kecil ini akan sangat penting ketika kita menghitung *running time* di Modul 03.

6.3 Dari pseudocode ke program

Berikut terjemahan kedua algoritma ke dalam Python. Perhatikan penyesuaian indeks: Python memulai indeks dari 0.

Python: terjemahan BILANGAN_MAX dan JUMLAH_n_BIL

```python
def bilangan_max(bil1, bil2, bil3):
    maks = bil1
    if maks < bil2:
        maks = bil2
    if maks < bil3:
        maks = bil3
    return maks

def jumlah_n_bil(A):
    jumlah = 0
    for i in range(len(A)):      # i = 0, 1, ..., n-1
        jumlah = jumlah + A[i]
    return jumlah

print(bilangan_max(7, 4, 10))
print(bilangan_max(9, 3, 5), bilangan_max(2, 8, 1), bilangan_max(4, 4, 4))
print(jumlah_n_bil([20, 35, 15]))
```

Keluaran program

```
10
9 8 4
70
```

ANALISIS DAN HUBUNGAN KONSEP

7. Bagaimana Konsep Ini Terhubung dengan Modul Lain

Karakteristik algoritma menentukan kebenarannya. *Finiteness* menjamin algoritma berhenti, sedangkan *output* dan *definiteness* menjamin hasil yang diberikan jelas dan dapat diperiksa. Tanpa ketiganya, kita bahkan tidak dapat mulai membicarakan seberapa cepat algoritma berjalan.

Struktur menentukan biaya. Struktur runtunan dan percabangan menjalankan setiap langkah paling banyak sekali, sehingga waktunya tidak bergantung pada banyaknya data. Sebaliknya, struktur perulangan menjalankan isi perulangan berkali-kali. Pada `JUMLAH_n_BIL`, banyaknya pengulangan sama dengan banyaknya data n. Jika n menjadi dua kali lipat, pekerjaannya juga menjadi kira-kira dua kali lipat. Pengamatan sederhana ini adalah inti analisis *running time* (Modul 03), dan pengelompokan laju pertumbuhannya dibahas di Modul 05–07.

Efektif belum tentu efisien. Untuk satu masalah biasanya ada banyak algoritma yang benar. Misalnya, ada banyak cara mengurutkan bilangan, seperti *Insertion Sort* (Modul 04) dan *Merge Sort* (Modul 08). Keduanya benar, tetapi kecepatannya sangat berbeda ketika datanya besar. Memilih algoritma yang tepat sering jauh lebih berpengaruh daripada membeli komputer yang lebih cepat. Klaim ini akan dibuktikan dengan perhitungan di Modul 05.

Pseudocode adalah bahasa bersama. Semua algoritma di modul-modul berikutnya ditulis dalam pseudocode dengan konvensi yang sama (`=` untuk pemberian nilai, `==` untuk perbandingan, indentasi untuk blok, indeks array dimulai dari 1).

KESIMPULAN

8. Kesimpulan

- Algoritma adalah urutan langkah logis dan terbatas yang mengubah masukan menjadi keluaran. Algoritma disebut benar bila untuk setiap masukan yang sah ia berhenti dengan keluaran yang benar.
- Program adalah wujud algoritma dalam bahasa pemrograman tertentu. Pemrograman adalah proses membuat, menguji, dan memperbaiki program.
- Enam karakteristik algoritma: *finiteness*, *input* (nol atau lebih), *output* (satu atau lebih), *definiteness*, *effectiveness*, dan *structured*. Efisiensi adalah pertimbangan terpisah yang diukur melalui analisis *running time*.
- Algoritma dapat ditulis secara naratif, dengan flowchart, atau dengan pseudocode. Pseudocode paling presisi dan menjadi standar dalam modul ini.
- Tiga struktur dasar algoritma adalah runtunan, percabangan, dan perulangan. Perulanganlah yang membuat waktu eksekusi bergantung pada ukuran masukan.
- Penelusuran (*tracing*) dengan berbagai masukan adalah cara praktis untuk memeriksa dan memahami algoritma.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Manakah pernyataan yang paling tepat tentang hubungan algoritma dan program?

A. Algoritma adalah program yang sudah dikompilasi.
B. Satu algoritma hanya dapat diterjemahkan ke satu bahasa pemrograman.
C. Program adalah wujud konkret algoritma dalam suatu bahasa pemrograman.
D. Algoritma dan program adalah istilah yang sepenuhnya sama.

2 ANALISIS KASUS

Perhatikan langkah berikut: "1) Isi `x` dengan 10. 2) Selama `x` tidak sama dengan 0, kurangi `x` dengan 3." Karakteristik algoritma manakah yang dilanggar? Jelaskan dengan menelusuri nilai `x`.

3 BENAR/SALAH DENGAN ALASAN

"Sebuah algoritma wajib memiliki minimal satu masukan." Benar atau salah? Berikan alasan dan contoh.

4 PILIHAN GANDA

Dalam konvensi pseudocode modul ini, apa perbedaan `a = b` dan `a == b`?

A. Keduanya sama-sama memeriksa apakah `a` sama dengan `b`.
B. `a = b` mengisi `a` dengan nilai `b`; `a == b` memeriksa kesamaan dan menghasilkan TRUE/FALSE.
C. `a = b` memeriksa kesamaan; `a == b` mengisi nilai.
D. `a == b` hanya dipakai di dalam perulangan `for`.

5 PENELUSURAN

Telusuri `BILANGAN_MAX(5, 12, 8)`. Tuliskan nilai `max` setelah setiap baris dan keluaran akhirnya. Berapa kali perintah pemberian nilai ke `max` dijalankan?

6 PENELUSURAN

Jalankan `JUMLAH_n_BIL` untuk $A = [4, -2, 9, 1]$. (a) Berapa keluarannya? (b) Berapa kali baris `for` diperiksa dan berapa kali baris `jumlah = jumlah + A[i]` dijalankan?

7 MERANCANG ALGORITMA

Tuliskan pseudocode `BILANGAN_MIN(bil1, bil2, bil3)` yang mengembalikan bilangan terkecil dari tiga bilangan. Sebutkan struktur dasar apa saja yang kamu gunakan.

8 PILIHAN GANDA

Sebuah algoritma mengurutkan n data dengan mencoba seluruh $n!$ kemungkinan susunan lalu memilih susunan yang urut. Pernyataan yang paling tepat adalah ...

A. Algoritma ini tidak memenuhi *finiteness*.
B. Algoritma ini benar dan berhenti, tetapi sangat tidak efisien untuk n besar.
C. Algoritma ini tidak memenuhi *output*.
D. Algoritma ini efisien karena pasti menemukan jawaban.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: C. Program adalah terjemahan algoritma ke bahasa pemrograman tertentu. Pilihan B salah karena satu algoritma dapat diterjemahkan ke banyak bahasa. Pilihan A dan D mencampuradukkan rancangan (algoritma) dengan wujudnya (program).

Soal 2

Melanggar *finiteness*. Nilai `x` berturut-turut 10, 7, 4, 1, −2, −5, ... Nilai `x` melompati 0 dan tidak pernah sama dengan 0, sehingga kondisi "`x` tidak sama dengan 0" selalu benar dan perulangan tidak pernah berhenti. Perbaikannya, misalnya, mengganti kondisi menjadi "selama `x > 0`".

Soal 3

Salah. Algoritma memiliki **nol** atau lebih masukan. Contoh algoritma tanpa masukan: mencetak bilangan 1 sampai 10, atau mencetak sepuluh bilangan prima pertama. Yang wajib minimal satu adalah keluaran.

Soal 4

Jawaban: B. Satu tanda sama dengan adalah pemberian nilai (*assignment*). Dua tanda sama dengan adalah perbandingan yang menghasilkan nilai logika TRUE atau FALSE, dan dipakai di dalam kondisi `if` atau `while`.

Soal 5

Baris 1: `max = 5`. Baris 2: $5 < 12$ TRUE, sehingga baris 3 memberi `max = 12`. Baris 4: $12 < 8$ FALSE, baris 5 dilewati. Baris 6 mengembalikan **12**. Pemberian nilai ke `max` terjadi **2 kali** (baris 1 dan baris 3).

Soal 6

(a) Nilai `jumlah` berturut-turut 0, 4, 2, 11, 12, sehingga keluarannya **12**. (b) Baris `for` diperiksa **5 kali** ($i = 1, 2, 3, 4$ bernilai benar, ditambah $i = 5$ yang menghentikan perulangan). Isi perulangan dijalankan **4 kali**. Secara umum, untuk n data, baris `for` diperiksa $n + 1$ kali dan isinya dijalankan n kali.

Soal 7

Salah satu jawaban yang benar:

BILANGAN_MIN(bil1, bil2, bil3)

```
1  min = bil1
2  if (min > bil2)
3      then min = bil2
4  if (min > bil3)
5      then min = bil3
6  return min
```

Struktur yang digunakan: **runtunan** (langkah berurutan) dan **percabangan** (dua pernyataan `if`). Tidak ada perulangan karena jumlah bilangan tetap tiga.

Soal 8

Jawaban: B. Banyaknya susunan $n!$ memang besar tetapi tetap terbatas, sehingga algoritma pasti berhenti (*finiteness* terpenuhi) dan menghasilkan keluaran yang benar. Masalahnya ada pada efisiensi. Untuk $n = 20$ saja, $20! \approx 2{,}4 \times 10^{18}$ susunan. Ini contoh bahwa algoritma yang benar dan efektif belum tentu efisien.

Referensi

Sumber utama: PENGERTIAN ALGORITMA PEMROGRAMAN (bagian A–C) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)
3. Turing, A. M. (1937). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265. https://doi.org/10.1112/plms/s2-42.1.230